Franck Pachot, dbi services

Join Methods and 12c Adaptive Plans

In its quest to improve cardinality estimation, 12c has introduced Adaptive Execution Plans which deals with the cardinalities that are difficult to estimate before execution. Ever seen a hanging query because a nested loop join is running on millions of rows?

This is the point addressed by Adaptive Joins. But that new feature is also a good occasion to look at the four possible join methods available for years.

Nested, Hash, Merge... What is a join?

In a relational database, each business entity is stored in a separate table. That makes its strength (query from different perspectives) but involves an expensive operation when we need to combine rows from several tables: the join operation.

```
SQL> select * from DEPT , EMP where DEPT.DEPTNO=EMP.DEPTNO;
```

The Oracle legacy syntax lists all the tables in the FROM clause, which, from a logical point of view only, does a cartesian product (combine each row from one source to each row from the other source) and then apply the join condition defined in the WHERE clause, as for any filtering predicates. The result – in the case of an inner join – is formed with the rows where the join condition evaluates to true.

Of course, this is not what is actually done or it would be very inefficient. The cartesian product multiplies the cardinalities from both sources before filtering, but the key to optimize a query is to filter as soon as possible before doing other operations.

```
SQL> select * from DEPT join EMP on DEPT.DEPTNO=EMP.DEPTNO;
```

The ANSI join syntax gives a better picture because it joins the tables one by one: the left table (that I will call the outer rowsource from now) is joined to the right table (the inner table) on a specific condition. That join condition involves some columns from both tables and it can be an equality (such as outer.colA=inner.colA) – this is an equijoin – or it can be an inequality (such as outer.colA>inner.colA) – known as thetajoin.

A variation is the semijoin (such as EXISTS) that do not duplicate the outer rows even if multiple inner rows are matching. And there is the antijoin (such as NOT IN) that returns only rows that do not match. In addition to that, an outer join will add some non-matching rows to an inner join.

Of course, the SQL syntax is declarative and will not determine the actual join order unless we force it with the LEADING hint.

The Oracle optimizer will choose the join order, joining an outer rowsource to an inner table, the outer rowsource being either the first table or the result from previous joins, aggregations etc.

And for each join the Oracle optimizer will choose the join method according to what is possible (Hash Join cannot do thetajoin for example) and the estimated cost. We can force another join method with hints as long as it is a possible one. When we want to force a join order and method, we need to set the order with the LEADING hint, listing all tables from outer to inner. Then for each inner table (all the ones in the LEADING hint except the first one) we will define the join method, distribution etc. Of course, this is not a recommendation for production. It is always better to improve cardinality estimations rather than restricting the CBO with hints. But sometimes we want to see and test a specific execution plan.

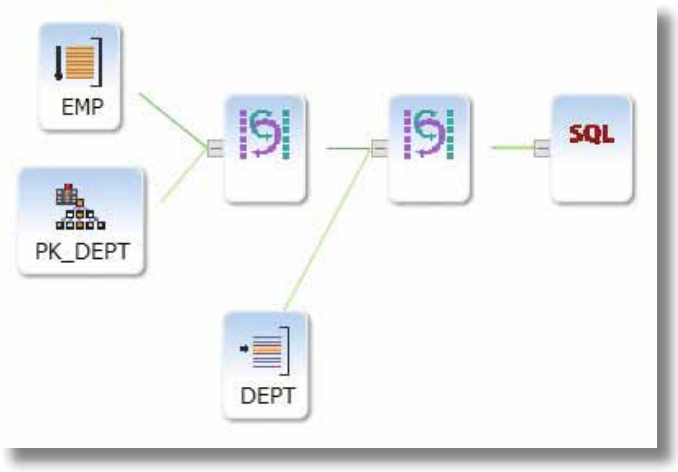
The execution plans in this article were retrieved after the execution in a session where statistics_level=all, using dbms_xplan:

```
select * from from table(dbms_xplan.display_cursor(format=>'allstats'))
```

This shows the execution plan for the last sql statement (we must be sure that no other statement is run in between, such as when serveroutput is set to on in sqlplus).

For better readability, I've reproduced only the relevant columns from executions plan.

Nested Loop Join



All join methods will read one rowsource and lookup for matching rows from the other table. What will differ among the different join methods is the structure used to access efficiently to that other table. And an efficient access usually involves sorting or hashing.

- Nested Loop will use a permanent structure access, such as an index that is already maintained sorted.
- Sort Merge join will build a sorted buffer from the inner table.
- Hash Join will build a hash table either from the inner table or the outer rowsource.
- Merge Join Cartesian will build a buffer from the inner table, which can be scanned several times.

Nested Loop is special in that it has nothing to do before starting to iterate over the outer rowsource. Thus, it's the most efficient way to return the first row quickly. But because it does nothing on the inner table before starting the loop, it requires an efficient way to access to the inner rows: usually an index access or hash cluster.

Here is the shape of the Nested Loop execution plan in 10g:

```
EXPLAINED SQL STATEMENT:
------------------------
select * from DEPT join EMP using(deptno) where sal>=3000

-----------------------------------------------------------------------------
| Id  | Operation                    | Name    | Starts | A-Rows | Buffers |
-----------------------------------------------------------------------------
|   0 | SELECT STATEMENT             |         |      1 |      3 |      13 |
|   1 |  NESTED LOOPS                |         |      1 |      3 |      13 |
|*  2 |   TABLE ACCESS FULL          | EMP     |      1 |      3 |       8 |
|   3 |   TABLE ACCESS BY INDEX ROWID| DEPT    |      3 |      3 |       5 |
|*  4 |    INDEX UNIQUE SCAN         | PK_DEPT |      3 |      3 |       2 |
-----------------------------------------------------------------------------
```

Execution Plan 1: Nested Loop Join in 10g

The inner table (EMP) had 3 rows (A-Rows=3) after applying the 'sal>=3000' predicate, and for each of them (Starts=3) the DEPT has been accessed by a rowid, which is coming from the index entry. The hints to force that join are: LEADING(EMP DEPT) for the join order and USE_NL(DEPT) to use Nested Loop when joining to the inner table DEPT. Both have to be used, or the CBO may choose another plan.

The big drawback of Nested Loop is the time to access to the inner table. Here we had to read 5 blocks (2 index branch+leaf and 2 table blocks) in order to retrieve only 3 rows. If the outer rowsource returns more than a few rows, then the Nested Loop is probably not an efficient method. This can be found on the execution plan with the 'A-Rows' from outer rowsource that determines the 'Starts' to access to the inner table.

The cost will be higher when the inner table is big (because of index depth) and when clustering factor is bad (more logical reads to access the table). It can be much lower when we don't need to access to the table at all (index having all required columns – known as covering index). But a nested loop that retrieves a lot of rows is always something expensive.

Since 9i, prefetching is done when accessing to the inner table in order to lower the logical reads. A further optimization has been introduced by 11g with Nested Loop Join Batching. A first loop retrieves a vector of rowid from the index and a second loop retrieves the table rows with a batched I/O (multiblock). It is faster but possible only when we don't need the result to be in the same order as the outer rowsource. Here is the plan in 11g:

```
-----------------------------------------------------------------------------
| Id  | Operation                    | Name    | Starts | A-Rows | Buffers |
-----------------------------------------------------------------------------
|   0 | SELECT STATEMENT             |         |      1 |      3 |      13 |
|   1 |  NESTED LOOPS                |         |      1 |      3 |      13 |
|   2 |   NESTED LOOPS               |         |      1 |      3 |      10 |
|*  3 |    TABLE ACCESS FULL         | EMP     |      1 |      3 |       8 |
|*  4 |    INDEX UNIQUE SCAN         | PK_DEPT |      3 |      3 |       2 |
|   5 |   TABLE ACCESS BY INDEX ROWID| DEPT    |      3 |      3 |       3 |
-----------------------------------------------------------------------------
```

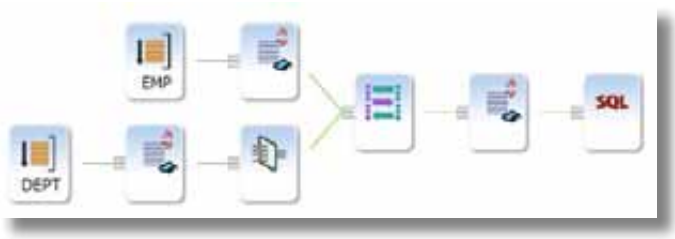
Execution Plan 2: Nested Loop Join in 11g

The 12c plan is the same except that it shows the following note: 'this is an adaptive plan'. We will see that later.

Nested Loop can be used for all types of joins and is efficient when we have few rows from outer rowsource that can be matched to the inner table through an efficient access path (usually index). It is the fastest way to retrieve the first rows quickly (pagination except when we need all rows before in order to sort them).

But when we have large tables to join, Nested Loop do not scale. The other join method that can do all kind of joins on large tables is the Sort Merge Join.

Sort Merge Join



Nested Loop can be used for all types of joins but is probably not optimal for inequalities, because of the cost of the range scan to access the inner table:

```
EXPLAINED SQL STATEMENT:
------------------------
select /*+ leading(EMP DEPT) use_nl(DEPT) index(DEPT) */ * from
DEPT join EMP on(EMP.deptno between DEPT.deptno and DEPT.deptno+1 )
where sal>=3000

-----------------------------------------------------------------------------
| Id  | Operation                    | Name    | Starts | A-Rows | Buffers |
-----------------------------------------------------------------------------
|   0 | SELECT STATEMENT             |         |      1 |      3 |      13 |
|   1 |  NESTED LOOPS                |         |      1 |      3 |      13 |
|   2 |   NESTED LOOPS               |         |      1 |      3 |      11 |
|*  3 |    TABLE ACCESS FULL         | EMP     |      1 |      3 |       8 |
|*  4 |    INDEX RANGE SCAN          | PK_DEPT |      3 |      3 |       3 |
|   5 |   TABLE ACCESS BY INDEX ROWID| DEPT    |      3 |      3 |       2 |
-----------------------------------------------------------------------------
```

Execution Plan 3: Nested Loop Join with inequality

Here I forced the plan to be a nested loop. But it can be very bad if the range scan returns a lot of rows.

So without hinting, a Sort Merge Sort is chosen by the CBO for that inequality join:

```
EXPLAINED SQL STATEMENT:
------------------------
select * from DEPT join EMP
on (EMP.deptno between DEPT.deptno and DEPT.deptno+10 ) where sal>=3000

-------------------------------------------------------------------------
| Id  | Operation             | Name | Starts | A-Rows | Buffers |  OMem |
-------------------------------------------------------------------------
|   0 | SELECT STATEMENT      |      |      1 |      5 |      14 |       |
|   1 |  MERGE JOIN           |      |      1 |      5 |      14 |       |
|   2 |   SORT JOIN           |      |      1 |      3 |       7 |  2048 |
|*  3 |    TABLE ACCESS FULL  | EMP  |      1 |      3 |       7 |       |
|*  4 |   FILTER              |      |      3 |      5 |       7 |       |
|*  5 |    SORT JOIN          |      |      3 |      5 |       7 |  2048 |
|   6 |     TABLE ACCESS FULL | DEPT |      1 |      4 |       7 |       |
-------------------------------------------------------------------------
```

Execution Plan 4: Sort Merge Join

The sort merge will use sorted rowsets in order to do the row matching without additional access. The outer rowsource is read and because both are sorted in the same way, it is easy to merge them. It is much quicker than a nested loop, but has an additional overhead: the rows must be sorted. The hints to force that plan are LEADING(EMP DEPT) USE_MERGE(DEPT)

In this example, both rowsources had to be sorted (SORT JOIN operation). This has an additional cost, and defers the first row retrieving because the SORT is a blocking operation (need to be completed before retrieving first result).

But when the outer row source is already ordered, and/or when the result must be ordered anyway, the Sort merge Join is an efficient method:

```
PLAN_TABLE_OUTPUT
EXPLAINED SQL STATEMENT:
------------------------
select * from DEPT join EMP using(deptno) where sal>=3000 order by deptno

-------------------------------------------------------------------------
| Id  | Operation             | Name | Starts | A-Rows | Buffers |  OMem |
-------------------------------------------------------------------------
|   0 | SELECT STATEMENT      |      |      1 |      3 |      11 |       |
|   1 |  MERGE JOIN           |      |      1 |      3 |      11 |       |
|*  2 |   TABLE ACCESS BY INDEX
          ROWID| EMP          |      1       |      3 |       4 |       |
|   3 |    INDEX FULL SCAN    | EMP_DEPTNO |1 |     14 |       2 |       |
|*  4 |   SORT JOIN           |      |      3 |      3 |       7 |  2048 |
|   5 |    TABLE ACCESS FULL  | DEPT |      1 |      4 |       7 |       |
-------------------------------------------------------------------------
```

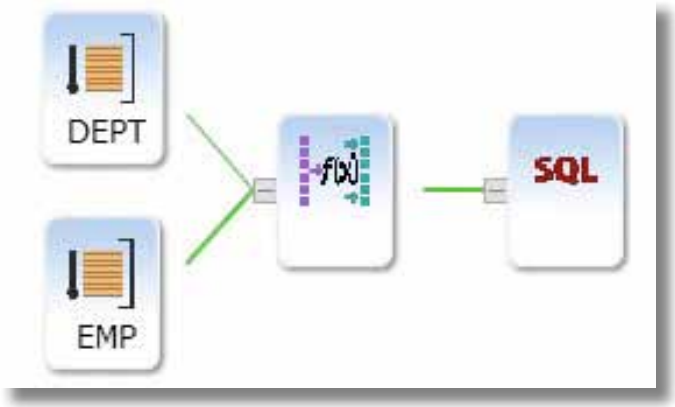
Execution Plan 5: Sort Merge Join without sorting

Here because I have an index on EMP.DEPTNO, which is ordered, there is no need to sort the outer rowsource. And there is no need to sort the result of the join because the Sort Merge Join returns rows in the same order as required by ORDER BY.

However, even when already sorted, the inner rowsource must always have a SORT JOIN operation, probably because only that sort structure (in memory or tempfile) has the ability to navigate backward when merging. So that join method always have an overhead before being able to return rows, even with sorted rowsources as input.

Merge join is very good for large rowsources, when one rowsource is already ordered, and when we need an ordered result. For example when joining multiple tables on the same join predicate, one sort will benefit to all joins. It is the only efficient join method for inequality joins on large tables. But when we do an equijoin, hashing is probably a faster algorithm than sorting.

Hash Join



We have seen that Sort Merge Join is used on large tables because Nested Loop is not scalable. The problem was that when we have n rows from the outer rowsource then the Nested Loop has to access the inner table n times, and each can involve 2 or 3 blocks through an index access. The best we can do – in the case of equijoin - is to access through a Single Table Hash Cluster where each access requires only one block to read. But Hash clusters as a permanent structure is difficult to maintain for large tables especially when the number of rows is difficult to predict. This is why it is not used as much as indexes.

But in a similar way to the Sort Merge Join that does sorting at each execution instead of accessing an ordered index, the Hash Join can build a hash table at each execution. The smallest rowset is used to build that, and then can be probed efficiently a large number of times.

Here is an example of a Hash Join plan:

```
EXPLAINED SQL STATEMENT:
------------------------
select * from DEPT join EMP using(deptno)

-------------------------------------------------------------------------------
| Id  | Operation             | Name | Starts | A-Rows | Buffers |  OMem |
-------------------------------------------------------------------------------
|   0 | SELECT STATEMENT      |      |      1 |     14 |      15 |       |
|*  1 |  HASH JOIN            |      |      1 |     14 |      15 |  1321K |
|   2 |   TABLE ACCESS FULL   | DEPT |      1 |      4 |       7 |       |
|   3 |   TABLE ACCESS FULL   | EMP  |      1 |     14 |       8 |       |
-------------------------------------------------------------------------------
```

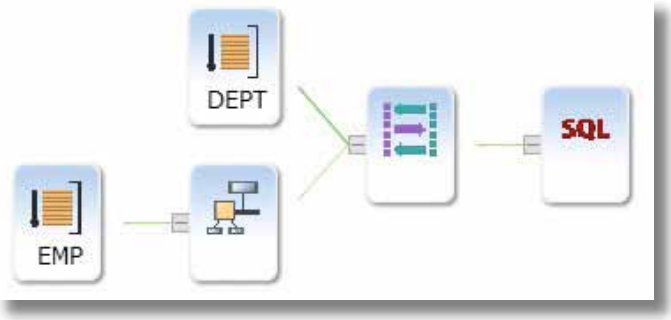
Execution Plan 6: Hash Join

Note that this time DEPT is above EMP. With Hash Join, inner and outer rowsource can be swapped. The smaller will be used to build the hash table and is shown above the driving table.

The plan above, where EMP is still the outer rowsource, is obtained with the following hints: LEADING(EMP DEPT) USE_HASH(DEPT) and SWAP_JOIN_INPUTS(DEPT) telling that DEPT – the inner table – will go above to be the built (hash) table. We can read those hints as: we have EMP, we join it to DEPT using a Hash Join and DEPT is the built table.

If we want EMP to be the built table, of course we can declare the opposite hints: LEADING(DEPT EMP) USE_HASH(EMP) SWAP_JOIN_INPUTS(EMP). But when we have more than 2 tables we cannot just change the LEADING order. Then, we will use the NO_SWAP_JOIN_INPUTS to let the outer rowsource be the built table.

Merge Join Cartesian



There is another join method that we see rarely. All the join methods we have seen above have an overhead coming from the access to the inner table: follow the index tree, sort, hash etc. What if both tables are so small that we prefer to scan it as a whole instead of searching an elaborated access path? We can imagine doing a Nested Loop from a small outer rowsource and FULL SCAN the inner table for each loop. But there is better: FULL SCAN once, put it in a buffer and then read the whole buffer for each outer rowsource row.

This is the Merge Join Cartesian:

```
EXPLAINED SQL STATEMENT:
------------------------
select /*+ leading(DEPT EMP) use_merge_cartesian(EMP) */ * from
DEPT join EMP using(deptno) where sal>3000

-------------------------------------------------------------------------------
| Id  | Operation             | Name | Starts | E-Rows | Buffers |  OMem |
-------------------------------------------------------------------------------
|   0 | SELECT STATEMENT      |      |      1 |        |      15 |       |
|   1 |  MERGE JOIN CARTESIAN |      |      1 |      1 |      15 |       |
|   2 |   TABLE ACCESS FULL   | DEPT |      1 |      4 |       8 |       |
|   3 |   BUFFER SORT         |      |      4 |      1 |       7 |  2048 |
|*  4 |    TABLE ACCESS FULL  | EMP  |      1 |      1 |       7 |       |
-------------------------------------------------------------------------------
```

Execution Plan 7: Merge join Cartesian

Now the outer rowsource is DEPT, we read 4 rows from it ('A-Rows'). For each of them we read the buffer ('Starts'=4) but the FULL TABLE SCAN occurred only the first time ('Starts'=1). Note that BUFFER SORT – despites its name – do not do any sorting. It is just buffered. The hints for that are LEADING(EMP DEPT) USE_MERGE_CARTESIAN(DEPT).

That is a good join method when the inner table is small but the join cardinality is high (e.g when for each outer row most of inner rows will match).

12c new feature: Adaptive Join

We have seen at the beginning that in 12c a Nested Loop was chosen by the optimizer, but mentioning that the plan is adaptive. Here is the plan with the '+adaptive' format of the dbms_xplan:

```
EXPLAINED SQL STATEMENT:
------------------------
select * from DEPT join EMP using(deptno) where sal>=3000

-----------------------------------------------------------------------------
|   Id  | Operation                  | Name    | Starts | A-Rows | Buffers |
-----------------------------------------------------------------------------
|     0 | SELECT STATEMENT           |         |      1 |      3 |      12 |
|- *  1 |  HASH JOIN                 |         |      1 |      3 |      12 |
|     2 |   NESTED LOOPS             |         |      1 |      3 |      12 |
|     3 |    NESTED LOOPS            |         |      1 |      3 |       9 |
|-    4 |     STATISTICS COLLECTOR   |         |      1 |      3 |       7 |
|  *  5 |      TABLE ACCESS FULL     | EMP     |      1 |      3 |       7 |
|  *  6 |     INDEX UNIQUE SCAN      | PK_DEPT |      3 |      3 |       2 |
|     7 |    TABLE ACCESS BY         |         |        |        |         |
|       |     INDEX ROWID            | DEPT    |      3 |      3 |       3 |
|-    8 |   TABLE ACCESS FULL        | DEPT    |      0 |      0 |       0 |
-----------------------------------------------------------------------------
```

Execution Plan 8: Adaptive Join resolved to Nested Loop

Most of the time, with an equijoin, the choice between Nested Loop and Hash Join will depend on the size of the outer rowsource.

If it is overestimated, then we risk doing a Hash join with a Full Table Scan of the inner table – that is not efficient when we match only few rows. But underestimating can be worse: the risk is to do millions of nested loops to access a table.

So 12c defers that choice to the first execution time, where a STATISTICS COLLECTOR will buffer and count the rows coming from the outer rowsource until it reaches the threshold cardinality.

Then if reached, it will switch to a Hash Join (the outer rowsource becoming the built table). The threshold is calculated at parse time where the CBO calculates the cost for Nested Loop and Hash Join for different cardinalities (dichotomy until Nested Loop cost is higher than Hash join cost).

The inflection point can be seen in the optimizer trace (event 10053). In my example it shows:

```
Found point of inflection for NLJ vs. HJ: card = 9.30
```

meaning that execution will switch to Hash Join when more than 10 rows are returned from EMP.

Here is the execution plan for the same query where the actual number of rows is 1000 instead of 3

```
EXPLAINED SQL STATEMENT:
------------------------
select * from DEPT join EMP using(deptno) where sal>=3000

-------------------------------------------------------------------------------
|   Id  | Operation                | Name       | Starts| A-Rows | Buffers|
-------------------------------------------------------------------------------
|     0| SELECT STATEMENT          |            |      1|      3 |     14 |
|  *  1|  HASH JOIN                |            |      1|   1000 |     14 |
|-    2|   NESTED LOOPS            |            |      1|   1000 |      7 |
|-    3|    NESTED LOOPS           |            |      1|   1000 |      7 |
|-    4|     STATISTICS COLLECTOR  |            |      1|   1000 |      7 |
|  *  5|      TABLE ACCESS FULL    | EMP        |      1|   1000 |      7 |
|- *  6|     INDEX UNIQUE SCAN     | PK_DEPT    |      0|      0 |      0 |
|-    7|    TABLE ACCESS BY
             INDEX ROWID            | DEPT       |      0|      0 |      0 |
|     8|   TABLE ACCESS FULL       | DEPT       |      1|      4 |      7 |
-------------------------------------------------------------------------------
```

Execution Plan 9: Adaptive Join resolved to Hash Join

The first 10 rows have been buffered, and then the nested loop has been discarded in favor of hash join. That inactivates the 1000 table access by index that would be necessary in nested loop.

Note that the choice occurs only on the first execution. Once the first execution has been done, the same plan will be used by the subsequent executions. We can see than in V$SQL:

- IS_RESOLVED_ADAPTIVE_PLAN=N when plan is adaptive but first execution did not occur yet
- IS_RESOLVED_ADAPTIVE_PLAN=Y when the choice has been done – after EXECUTIONS >= 1

So this is a solution when estimation was bad, but not a solution when data changes. Another 12c feature, Automatic Reoptimization, is there to adapt for subsequent executions.

Conclusion

Each join method has its use cases where it is efficient:

- Nested Loop Join is good when we want to retrieve few rows for which we have a fast access path. This is typical of OLTP. Proper indexing is the key to good performance.
- Hash Join is good when we have large tables, typical of reporting or BI, especially when the smallest table fits in memory. But it is available only for equijoins.
- Merge Join Cartesian should be seen only with small tables and with a high multiplicity for the join.
- Sort Merge Join can be seen when the sorting is not a big overhead, or for thetajoins where hash join is not possible.

About resources, Nested Loops is more about indexes, buffer cache (SGA), flash cache and single block reads. Sort Merge and Hash Joins are more about full table scan, multi-block reads, direct reads/smartscan and workarea (PGA) and tempfiles.

The choice among the possible methods depends mainly on the size of the tables. The estimation can be quite good for a two table join with accurate object statistics, and/or dynamic sampling. But after several joins and predicate filters, the error made by the optimizer may be large, leading to an inefficient execution plan.

Some cardinality estimations can be more accurate at execution time. This is why 10g introduced bind variable peeking, and 11g came with Adaptive Cursor Sharing. And now 12c goes a step further with Adaptive Plans where a plan chosen for small rowsource can be adapted when the actual number of rows is higher than expected. ■

Contact

dbi services

Franck Pachot
E-Mail:
franck.pachot@dbi-services.com

Franck Pachot, dbi services

Parallel Distribution and 12c Adaptive Plans

In the previous newsletter we have seen how 12c can defer the choice of the join method to the first execution. We considered only serial execution plans. But besides join method, the cardinality estimation is a key decision for parallel distribution when joining in parallel query. Ever seen a parallel query consuming huge tempfile space because a large table is broadcasted to lot of parallel processes? This is the point addressed by Adaptive Parallel Distribution.

Once again, that new feature is a good occasion to look at the different distribution methods.

Parallel Query Distribution

I'll do the same query as in previous newsletter, joining EMP with DEPT, but now I choose to set a parallel degree 4 to the EMP table. If I do the same hash join as before, DEPT being the built table, I will have:

- Four consumer processes that will do the Hash Join.
- One process (the coordinator) reading DEPT which is not in parallel – and sending rows to one of the consumer processes, depending on the hash value calculated from on the join column values.
- Each of the four consumers receives their part of the DEPT rows and hash them to create their built table.
- Four producer processes, each reading specific granules of EMP, send each row to one of the four consumer.
- Each of the four consumers receives their part of EMP rows and matches them to their probe table.
- Each of them sends their result to the coordinator. Because the work was divided with a hash function on the join column, the final result of the join is just the concatenation of each consumer result.

Here is the execution plan for that join:

```
EXPLAINED SQL STATEMENT:
------------------------
select * from DEPT join EMP using(deptno)

-----------------------------------------------------------------------------------------------------------------
| Id  | Operation                | Name       |Starts |   TQ   |  IN-OUT| PQ Distrib | A-Rows |  Buffers |  OMem |
-----------------------------------------------------------------------------------------------------------------
|   0 | SELECT STATEMENT         |            |     1 |        |        |            |     14 |       10 |       |
|   1 |  PX COORDINATOR          |            |     1 |        |        |            |     14 |       10 |       |
|   2 |   PX SEND QC (RANDOM)    | :TQ10002   |     0 |  Q1,02 |   P->S |  QC (RAND) |      0 |        0 |       |
|*  3 |    HASH JOIN BUFFERED    |            |     4 |  Q1,02 |   PCWP |            |     14 |        0 |  1542K |
|   4 |     BUFFER SORT          |            |     4 |  Q1,02 |   PCWC |            |      4 |        0 |  2048 |
|   5 |      PX RECEIVE          |            |     4 |  Q1,02 |   PCWP |            |      4 |        0 |       |
|   6 |       PX SEND HASH       | :TQ10000   |     0 |        |   S->P |       HASH |      0 |        0 |       |
|   7 |        TABLE ACCESS FULL | DEPT       |     1 |        |        |            |      4 |        7 |       |
|   8 |    PX RECEIVE            |            |     3 |  Q1,02 |   PCWP |            |     14 |        0 |       |
|   9 |     PX SEND HASH         | :TQ10001   |     0 |  Q1,01 |   P->P |       HASH |      0 |        0 |       |
|  10 |      PX BLOCK ITERATOR   |            |     4 |  Q1,01 |   PCWC |            |     14 |       15 |       |
|* 11 |       TABLE ACCESS FULL  | EMP        |     5 |  Q1,01 |   PCWP |            |     14 |       15 |       |
-----------------------------------------------------------------------------------------------------------------
```

Execution Plan 1: PX hash distribution

The Q1,01 is the producer set that reads EMP, the Q1,02 is the consumer set that does the join. The 'PQ Distrib' column shows the HASH distribution for both the outer rowsource DEPT and the inner table EMP. The hint for that is PQ_DISTRIBUTE(DEPT HASH HASH) to be added to the leading(EMP DEPT) use_hash(DEPT) swap_join_inputs(DEPT) that defines the join order and method.

This is efficient when both tables are big. But with a DOP of 4 we have 1+2*4=8 processes and a lot of messaging among them.

When one table is not so big, then we can avoid a whole set of parallel processes. We can broadcast the small table (DEPT) to the 4 parallel processes doing the join. In that case, the same set of processes is able to read EMP and do the join.

Here is the execution plan:

```
EXPLAINED SQL STATEMENT:
------------------------
select /*+ leading(EMP DEPT) use_hash(DEPT) swap_join_inputs(DEPT) pq_distribute(DEPT NONE BROADCAST) */ * from
DEPT join EMP using(deptno)

-----------------------------------------------------------------------------------------------------------
| Id  | Operation                | Name     |Starts |    TQ |IN-OUT | PQ Distrib | A-Rows | Buffers |   OMem |
-----------------------------------------------------------------------------------------------------------
|   0 | SELECT STATEMENT         |          |     1 |       |       |            |     14 |      10 |        |
|   1 |  PX COORDINATOR          |          |     1 |       |       |            |     14 |      10 |        |
|   2 |   PX SEND QC (RANDOM)    | :TQ10001 |     0 | Q1,01 | P->S  | QC (RAND)  |      0 |       0 |        |
|*  3 |    HASH JOIN             |          |     4 | Q1,01 | PCWP  |            |     14 |      15 |   1321K|
|   4 |     BUFFER SORT          |          |     4 | Q1,01 | PCWC  |            |     16 |       0 |   2048 |
|   5 |      PX RECEIVE          |          |     4 | Q1,01 | PCWP  |            |     16 |       0 |        |
|   6 |       PX SEND BROADCAST  | :TQ10000 |     0 |       | S->P  | BROADCAST  |      0 |       0 |        |
|   7 |        TABLE ACCESS FULL | DEPT     |     1 |       |       |            |      4 |       7 |        |
|   8 |     PX BLOCK ITERATOR    |          |     4 | Q1,01 | PCWC  |            |     14 |      15 |        |
|*  9 |      TABLE ACCESS FULL   | EMP      |     5 | Q1,01 | PCWP  |            |     14 |      15 |        |
-----------------------------------------------------------------------------------------------------------
```

Execution Plan 2: PX broadcast from serial

The coordinator reads DEPT and broadcasts all rows to each parallel server process (Q1,01). Those processes build the hash table for DEPT and then read their granules of EMP.

With the PQ_DISTRIBUTE we can choose how to distribute a table to the consumer that will process the rows. The syntax is PQ_DISTRIBUTE(inner_table outer_distribution inner_distribution). For HASH we must use the same hash function, so we will see PQ_DISTRIBUTE(DEPT HASH HASH) for producers sending to consumer according to the hash function.

We can choose to broadcast the inner table with PQ_DISTRIBUTE(DEPT NONE BROADCAST) or the outer rowsource PQ_DISTRIBUTE(DEPT BROADCAST NONE). The broadcasted table will be received in a whole by each consumer, so it can take a lot of memory, when it is buffered by the join operation and when the DOP is high.

When the tables are partitioned, the consumers can divide their job by partitions instead of granules, and we can distribute rows that match each consumer partition. For example, if EMP is partitioned on DEPTNO, then PQ_DISTRIBUTE(DEPT NONE PARTITION) will distribute the DEPT rows to the right consumer process according to DEPTNO value. The opposite PQ_DISTRIBUTE (DEPT PARTITION NONE) would be done, if DEPT were partitioned on DEPTNO.

And if both EMP and DEPT are partitioned on DEPTNO, then there is nothing to distribute: PQ_DISTRIBUTE(DEPT NONE NONE) because each parallel process is able to read both EMP and DEPT partition and do the Hash Join. This is known as partition-wise join and is very efficient when the number of partition is equal to the DOP, or a large multiple.

12c Small Table Replicate

If we take the example above where DEPT was broadcasted, but setting a parallel degree on DEPT as well, we have the following execution plan:

```
-----------------------------------------------------------------------------------------------------------
| Id  | Operation                | Name     |Starts |    TQ  | IN-OUT | PQ Distrib | A-Rows | Buffers |  OMem |
-----------------------------------------------------------------------------------------------------------
|   0 | SELECT STATEMENT         |          |     1 |        |        |            |     14 |       6 |       |
|   1 |  PX COORDINATOR          |          |     1 |        |        |            |     14 |       6 |       |
|   2 |   PX SEND QC (RANDOM)    | :TQ10001 |     0 | Q1,01  |   P->S | QC (RAND)  |      0 |       0 |       |
|*  3 |    HASH JOIN             |          |     4 | Q1,01  |   PCWP |            |     14 |      15 |  1321K|
|   4 |     PX RECEIVE           |          |     4 | Q1,01  |   PCWP |            |     16 |       0 |       |
|   5 |      PX SEND BROADCAST   | :TQ10000 |     0 | Q1,00  |   P->P | BROADCAST  |      0 |       0 |       |
|   6 |       PX BLOCK ITERATOR  |          |     4 | Q1,00  |   PCWC |            |      4 |      15 |       |
|*  7 |        TABLE ACCESS FULL | DEPT     |     5 | Q1,00  |   PCWP |            |      4 |      15 |       |
|   8 |      PX BLOCK ITERATOR   |          |     4 | Q1,01  |   PCWC |            |     14 |      15 |       |
|*  9 |       TABLE ACCESS FULL  | EMP      |     5 | Q1,01  |   PCWP |            |     14 |      15 |       |
-----------------------------------------------------------------------------------------------------------
```

Execution Plan 3: PX broadcast from parallel

Here we have a set of producers (Q1,00) that will broadcast to all consumers (Q1,01). That was the behavior in 11g.

In 12c a step further than broadcasting can be done by replicating the reading of DEPT in all consumers instead of broadcasting.

```
-----------------------------------------------------------------------------------------------------------
| Id  | Operation                | Name     |Starts |    TQ  | IN-OUT | PQ Distrib | A-Rows | Buffers |  OMem |
-----------------------------------------------------------------------------------------------------------
|   0 | SELECT STATEMENT         |          |     1 |        |        |            |     14 |       3 |       |
|   1 |  PX COORDINATOR          |          |     1 |        |        |            |     14 |       3 |       |
|   2 |   PX SEND QC (RANDOM)    | :TQ10000 |     0 | Q1,00  |   P->S | QC (RAND)  |      0 |       0 |       |
|*  3 |    HASH JOIN             |          |     4 | Q1,00  |   PCWP |            |     14 |      43 |  1321K|
|   4 |     TABLE ACCESS FULL    | DEPT     |     4 | Q1,00  |   PCWP |            |     16 |      28 |       |
|   5 |     PX BLOCK ITERATOR    |          |     4 | Q1,00  |   PCWC |            |     14 |      15 |       |
|*  6 |      TABLE ACCESS FULL   | EMP      |     5 | Q1,00  |   PCWP |            |     14 |      15 |       |
-----------------------------------------------------------------------------------------------------------
```

Execution Plan 4: PQ replicate

That optimization requires more I/O (but it concerns only small tables anyway – in can be cached when using In-Memory parallel execution) but saves processes, memory and messaging. The hint is PQ_DISTRIBUTE(DEPT NONE BROADCAST) PQ_REPLICATE(DEPT)

12c Adaptive Parallel Distribution

12c comes with Adaptive Plans. We have seen in the previous newsletter the Adaptive Join when it is difficult to estimate the cardinality and to choose between Nested Loop and Hash Join. It is the same concern here when choosing between broadcast and hash distribution: Adaptive Parallel Distribution.

The previous HASH HASH parallel plans were done in 11g. Here is the same in 12c:

```
EXPLAINED SQL STATEMENT:
------------------------
select * from DEPT join EMP using(deptno)

-----------------------------------------------------------------------------------------------------------
| Id  | Operation                | Name     |Starts |    TQ  | IN-OUT | PQ Distrib | A-Rows | Buffers |  OMem |
-----------------------------------------------------------------------------------------------------------
|   0 | SELECT STATEMENT         |          |     1 |        |        |            |     14 |      10 |       |
|   1 |  PX COORDINATOR          |          |     1 |        |        |            |     14 |      10 |       |
|   2 |   PX SEND QC (RANDOM)    | :TQ10002 |     0 | Q1,02  |   P->S |  QC (RAND) |      0 |       0 |       |
|*  3 |    HASH JOIN BUFFERED    |          |     4 | Q1,02  |   PCWP |            |     14 |       0 |  1542K|
|   4 |     BUFFER SORT          |          |     4 | Q1,02  |   PCWC |            |     16 |       0 |  2048 |
|   5 |      PX RECEIVE          |          |     4 | Q1,02  |   PCWP |            |     16 |       0 |       |
|   6 |       PX SEND HYBRID HASH| :TQ10000 |     0 |        |   S->P |HYBRID HASH |      0 |       0 |       |
|   7 |        STATISTICS COLLECTOR|        |     1 |        |        |            |      4 |       7 |       |
|   8 |         TABLE ACCESS FULL| DEPT     |     1 |        |        |            |      4 |       7 |       |
|   9 |     PX RECEIVE           |          |     4 | Q1,02  |   PCWP |            |     14 |       0 |       |
|  10 |      PX SEND HYBRID HASH | :TQ10001 |     0 | Q1,01  |   P->P |HYBRID HASH |      0 |       0 |       |
|  11 |       PX BLOCK ITERATOR  |          |     4 | Q1,01  |   PCWC |            |     14 |      15 |       |
|* 12 |        TABLE ACCESS FULL | EMP      |     5 | Q1,01  |   PCWP |            |     14 |      15 |       |
-----------------------------------------------------------------------------------------------------------
```

Execution Plan 5: Adaptive Parallel Distribution

The distribution is HYBRID HASH and there is a STATISTICS COLLECTOR before sending to parallel server consumers. Oracle will count the rows coming from DEPT and will choose to BROADCAST or HASH depending on the number of rows.

It is easy to check what has been chosen here, knowing that the DOP was 4. I have 4 rows coming from DEPT ('A-rows' on DEPT TABLE ACCESS FULL) and 16 were received by the consumer ('A-Rows' on PX RECEIVE): this is broadcast (4x4=16).

Parallel Query Distribution from SQL Monitoring

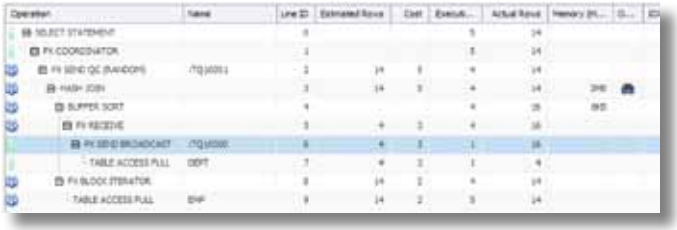
When we have the Tuning Pack, it is easier to get execution statistics from SQL Monitoring. Here are the same execution plans as above, but gathered with SQL Monitoring reports.

The coordinator in green does everything that is done in serial. The producers are in blue, the consumers are in red.

Here is the Hash distribution where DEPT read in serial and EMP read in parallel are both distributed to the right consumer that does the join:

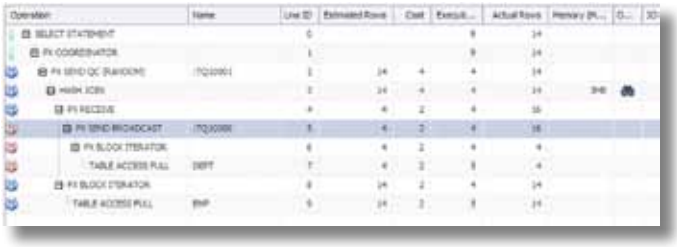
SQL Monitor 1: PX hash distribution

Here is the broadcast from DEPT serial read:



SQL Monitor 2: PX broadcast from serial

And the broadcast from DEPT parallel read (two sets of parallel servers):



SQL Monitor 3: PX broadcast from parallel

Then here is the 12c Small Table Replicate allowing to read DEPT from the same set of parallel processes that is doing the join:

SQL Monitor 4: PQ replicate

And in 12c, the choice between HASH and BROADCAST being done at runtime, and called HYBRID HASH:

SQL Monitor 5: Adaptive Parallel Distribution

Conclusion

Long before MapReduce became a buzzword, Oracle was able to distribute the processing of SQL queries to several parallel processes (and to several nodes when in RAC). Reading a table in parallel is easy: Each process reads a separate chunk. But when we need to join tables, then the rows have to be distributed from a set of producers (which full scan their chunks) to a set of consumers (which will do the join). Small row sets do not need to be processed in parallel and can be broadcasted to each consumer. But large rowset will be distributed to the right process only. The choice depends on the size and then the Cost Based Optimizer estimation of cardinality is a key point.

As we have seen for join methods, Oracle 12c can defer that choice to the first execution. This is Adaptive Parallel Distribution. ■

Contact

dbi services

Franck Pachot
E-Mail:
franck.pachot@dbi-services.com

Franck Pachot, dbi services

Star Transformation, 12c Adaptive Bitmap Pruning and In-Memory option

In the previous newsletters I've described Adaptive Plans, the 12c new feature where the CBO can generate multiple sub-plans and select the right one at the first execution time. And that was a pretext to describe join methods and parallel query distribution which are not always well known. But beside adaptive joins and adaptive parallel distribution, 12c comes with Adaptive Bitmap Pruning. So my articles become a trilogy and, as I did previously, I'll describe the case it applies to and which is often not well known: the Star Transformation.

Star schema

When a transactional application updates your data you store it in a structure that is close to what you insert: one tab-le per business entity, and relational integrity among them. And you query usually in the same way, joining few rows from several tables.

But when a database is dedicated to query, and queries in BI often involve lot of rows, you prefer to store them close to the way you retrieve the data. You put all the measures that are related to same information (and same granularity) in a FACT table. And around that table with lot of rows you put smaller tables with all information about the axes of analysis – known as DIMENSIONS.

This is the star schema that I prefer to call a dimensional model.

A query on a star schema involves:

- Several predicates on dimension attributes (ex: sales date between two dates, country code in a list). They are columns on the dimension tables. Dimension tables are small (e.g. countries) or medium (e.g. customer)
- One or several measures to be retrieved. They are columns in the fact table, usually numbers. The fact table has a lot of rows and we usually need to read a lot of rows and aggregate them later
- Additional information from the dimension table (e.g. display the country name whereas predicate was on country code)

The usual access path (index range scan filtering all predicates and then access to table to get the measures) is not optimal or not possible for two reasons:

- Having all predicates in the same index is not possible because we can't have an index for each possible predicate combination
- Adding the additional information in the table would make it very large.

So basically, we build a schema with:

- One FACT table that has the minimum of columns (because it's already big because of the number of rows). Only the dimension key and the measures
- Several DIMENSION tables that has the key, columns where you will have predicates on, and all other information. They can be large (lot of columns) because they don't have a huge number of rows and can be denormalized (have all hierarchy) because they are quite static.
- The dimension keys in the FACT table are declared as foreign key to their DIMENSION tables.

SMS >>>

Oracle Open World

Jedes Jahr wird die Berichterstattung der Oracle Openworld besser. Livestreams, Tickers usw machen einen virtuellen Besuch möglich. Für jene die nicht unbedingt bis spät in die Nacht das Geschehen verfolgen wollten sind Aufzeichnungen aller Keynotes, garniert mit weiteren Informationen, verfügbar (http://tinyurl.com/m2mpmq3). Das funktioniert sogar mit dem Smartphone (mit dem der QR Code rechts besonders viel Spass macht):

- Each dimension key in the FACT table has a bitmap index on it so that all predicate results can be merge quickly before goring to the large FACT table

Test case

I've build the following test case with one FACT table and three DIMENSION tables:

```
create table DIM1 as select rownum DIM1_ID , … DIM1_COD, … DIM1_TXT from dual connect by level<=10;
create table DIM2 as select DIM1_ID DIM2_ID,DIM1_COD DIM2_COD,DIM1_TXT DIM2_TXT from DIM1 where rownum<=10;
create table DIM3 as select DIM1_ID DIM3_ID,DIM1_COD DIM3_COD,DIM1_TXT DIM3_TXT from DIM1 where rownum<=10;
```

Those are my 3 dimension tables with an ID (the dimension key), a COD (where I'll have some critera on) and a TXT (the additional information). I've 10 rows in each.

```
create table FACT as select rownum FACT_ID,DIM1_ID,DIM2_ID,DIM3_ID,mod(rownum,1000)/10 FACT_MESURE from
DIM1,DIM2,DIM3,(select * from dual connect by level<=1000);
```

This is my FACT table. I have 1000 rows per each combination of dimensions, so 1 million rows.

And I define the primary keys on the DIMENSION table and the foreign keys on the FACT table, as well as an index bitmap for each foreign key.

```
alter table DIM1 add constraint DIM1PK primary key(DIM1_ID);
alter table FACT add constraint DIM1FK foreign key (DIM1_ID) references DIM1;
create index DIM1BX on FACT(DIM1_ID);
alter table DIM2 add constraint DIM2PK primary key(DIM2_ID);
alter table FACT add constraint DIM2FK foreign key (DIM2_ID) references DIM2;
create index DIM2BX on FACT(DIM2_ID);
alter table DIM3 add constraint DIM3PK primary key(DIM3_ID);
alter table FACT add constraint DIM3FK foreign key (DIM3_ID) references DIM3;
create index DIM3BX on FACT(DIM3_ID);
```

Finally I gather statistics and, in order to simulate one large dimension, I fake the stats for DIM1 as if it has 100'000 rows:

```
exec dbms_stats.gather_schema_stats(user);
exec dbms_stats.set_table_stats(user,'DIM1',numrows=>1e5);
```

And now it's time to check some execution plans. I'm running the following query:

```
explain plan for select * from FACT
 join DIM1 using(DIM1_ID)
 join DIM2 using(DIM2_ID)
 join DIM3 using(DIM3_ID)
 where DIM1_COD='One' and DIM2_COD='One' and DIM3_COD='One';
```

That is:

- Predicate on DIM1, DIM2, DIM3 columns
- All measures from FACT
- Additional information from DIM1, DIM2, DIM3

Without star transformation

Here is the execution plan when I leave the 'star_transformation_enabled' to its default which is false:

```
NAME                                 TYPE         VALUE
------------------------------------ ------------ ----------
star_transformation_enabled          string       FALSE
```

I use SQL Monitor which, in 12.1.0.2, shows adaptive plans, having the inactive part in gray:

Operation	Name	Exec...	Actual Rows
SELECT STATEMENT		1	1
HASH JOIN		1	1
NESTED LOOPS		1	1,000
NESTED LOOPS		1	1,000
STATISTICS COLLECTOR		1	1
MERGE JOIN CARTESIAN		1	1
MERGE JOIN CARTESIAN		1	1
TABLE ACCESS FULL	DIM2	1	1
BUFFER SORT		1	1
TABLE ACCESS FULL	DIM3	1	1
BUFFER SORT		1	1
TABLE ACCESS FULL	DIM1	1	1
BITMAP CONVERSION TO ROWIDS		1	1,000
BITMAP AND		1	1
BITMAP INDEX SINGLE VALUE	DIM1BX	1	6
BITMAP INDEX SINGLE VALUE	DIM2BX	1	12
BITMAP INDEX SINGLE VALUE	DIM3BX	1	40
TABLE ACCESS BY INDEX ROWID	FACT	1,000	1,000
TABLE ACCESS FULL	FACT	0	0

That's a long plan but not so complex. Here is what it does:

- First it reads all the dimensions DIM1, DIM2 and DIM3 (each one filtered with its own predicate) and does a cartesian join to get all the combination that are allowed by our predicates. This resultset has the dimension key to get to the FACT and has also the additional information we need for the final result.
- Then the STATISTICS COLLECTOR will decide on the sub-plan to choose (this is 12c adaptive join as I described in the previous newsletter).
- If the number of combination is not too large, it will do a NESTED LOOP: for each combination we get to the matching FACT rows. This is done through the bitmap indexes: for each dimension key, the corresponding bitmap index is accessed (BITMAP INDEX SINGLE VALUE), giving a bitmap of rows which are merge (BITMAP AND) and converted to ROWID. Then with those ROWID we loop to access to the FACT table.
- If the number of combination is large, then it is better to full scan the FACT table and do the join with the dimension combination through a HASH JOIN.

Star transformation without temporary table

Let's enable star transformation:

```
SQL> alter session set star_transformation_enabled=temp_disable;
Session altered.
```

Yes, there is no mistake here. Star transformation is enabled but without 'temp' which we will see later.

Do you remember that I've described star queries with two accesses to dimensions? One to apply the predicate (and get the dimension key for the result) and the other one to get the additional information once we got the FACT rows.

The principle of STAR transformation is to push the first one as if it were and IN (SELECT ID from DIM WHERE ...)

So let's look at the plan:

Operation	Name	Exe...	Actual Rows
SELECT STATEMENT		1	1,000
HASH JOIN		1	1,000
MERGE JOIN CARTESIAN		1	1
MERGE JOIN CARTESIAN		1	1
TABLE ACCESS FULL	DIM2	1	1
BUFFER SORT		1	1
TABLE ACCESS FULL	DIM3	1	1
BUFFER SORT		1	1
TABLE ACCESS FULL	DIM1	1	1
VIEW	VW_ST_A	1	1,000
NESTED LOOPS		1	1,000
BITMAP CONVERSION TO ROWIDS		1	1,000
BITMAP AND		1	1
BITMAP MERGE		1	1
BITMAP KEY ITERATION		1	6
TABLE ACCESS FULL	DIM1	1	1
BITMAP INDEX RANGE SCAN	DIM1BX	1	6
STATISTICS COLLECTOR		1	2
BITMAP MERGE		1	2
BITMAP KEY ITERATION		1	12
TABLE ACCESS FULL	DIM2	1	1
BITMAP INDEX RANGE SCAN	DIM2BX	1	12
STATISTICS COLLECTOR		1	5
BITMAP MERGE		1	5
BITMAP KEY ITERATION		1	40
TABLE ACCESS FULL	DIM3	1	1
BITMAP INDEX RANGE SCAN	DIM3BX	1	40
TABLE ACCESS BY USER ROWID	FACT	1,000	1,000

The first part – the MERGE JOIN CARTESIAN – is similar, but now the BITMAP INDEX SINGLE VALUE has been replaced. We read the dimension, apply the predicate, and for each dimension key we get to the bitmap index (BITMAP INDEX RANGE SCAN). The bitmaps are then merged for each dimension (BITMAP MERGE) and then ANDed with the ones coming from the other dimensions.

This is very efficient when the predicate has a good selectivity.

But what if the dimension is a big table? We have to read it two times here.

Star transformation with temporary table

Let's enable star transformation with temporary table:

```
SQL> alter session set star_transformation_enabled=true;
Session altered.
```

Remember, I've set the stats so that DIM1 appears as a large dimension. In order to avoid to read it two times, the optimizer can choose to put it in a temporary table. Look at the beginning of the plan:

The DIM1 table is first loaded as a temporary table. Then that temporary table is used in the dimension merge cartesian join (to build the hash table to lookup for the additional information) and is also used to apply the predicate before going to the DIM1BIX bitmap index:

Operation	Name	Ex...	Actual Rows
SELECT STATEMENT		1	1,000
TEMP TABLE TRANSFORMATION		1	1,000
LOAD AS SELECT		1	2
TABLE ACCESS FULL	DIM1	1	1
HASH JOIN		1	1,000
MERGE JOIN CARTESIAN		1	1
MERGE JOIN CARTESIAN		1	1
TABLE ACCESS FULL	DIM2	1	1
BUFFER SORT		1	1
TABLE ACCESS FULL	SYS_TEMP_	1	1
BUFFER SORT		1	1
TABLE ACCESS FULL	DIM3	1	1
VIEW	VW_ST_B7	1	1,000
NESTED LOOPS		1	1,000
BITMAP CONVERSION TO ROWIDS		1	1,000
BITMAP AND		1	1
BITMAP MERGE		1	1
BITMAP KEY ITERATION		1	6
TABLE ACCESS FULL	SYS_TEMP_	1	1
BITMAP INDEX RANGE SCAN	DIM1BX	1	6
STATISTICS COLLECTOR		1	2
BITMAP MERGE		1	2
BITMAP KEY ITERATION		1	12
TABLE ACCESS FULL	DIM2	1	1
BITMAP INDEX RANGE SCAN	DIM2BX	1	12
STATISTICS COLLECTOR		1	5
BITMAP MERGE		1	5
BITMAP KEY ITERATION		1	40
TABLE ACCESS FULL	DIM3	1	1
BITMAP INDEX RANGE SCAN	DIM3BX	1	40
TABLE ACCESS BY USER ROWID	FACT	1,000	1,000

Nothing else different. It's the same principle:

- Join each dimension to the FACT bitmap indexes
- Get the resulting ROWIDs and get the FACT rows
- Then join back to the dimension cartesian join in order to get additional information.

12c Adaptive Bitmap Pruning

So, all that exists before 12c. What is new is that grayed 'STATISTICS COLLECTOR'. I said that star transformation is good when the predicate is selective enough to filter few rows. Imagine that the cardinality estimation was wrong, and most of FACT rows have the required value. Then the optimizer can choose to stop iterating in that bitmap branch. We just ignore the predicate at that step, and the join back to the dimension Cartesian join will filter it anyway.

If you check the execution plan with predicates, you will see the predicate on dimension in the two table access.

Here I still run the same query but I've changed my data. In the previous examples, only 1 row was coming out from the DIM3 dimension (Actual Rows in the execution plan). Now I have 6 rows in DIM3 that are returned:

Operation	Name	Ex...	Actual Rows
SELECT STATEMENT		1	6,000
TEMP TABLE TRANSFORMATION		1	6,000
LOAD AS SELECT		1	2
TABLE ACCESS FULL	DIM1	1	1
HASH JOIN		1	6,000
MERGE JOIN CARTESIAN		1	6
MERGE JOIN CARTESIAN		1	1
TABLE ACCESS FULL	DIM2	1	1
BUFFER SORT		1	1
TABLE ACCESS FULL	SYS_TEMP_	1	1
BUFFER SORT		1	6
TABLE ACCESS FULL	DIM3	1	6
VIEW	VW_ST_B7	1	6,000
NESTED LOOPS		1	6,000
BITMAP CONVERSION TO ROWIDS		1	6,000
BITMAP AND		1	1
BITMAP MERGE		1	1
BITMAP KEY ITERATION		1	6
TABLE ACCESS FULL	SYS_TEMP_	1	1
BITMAP INDEX RANGE SCAN	DIM1BX	1	6
STATISTICS COLLECTOR		1	2
BITMAP MERGE		1	2
BITMAP KEY ITERATION		1	12
TABLE ACCESS FULL	DIM2	1	1
BITMAP INDEX RANGE SCAN	DIM2BX	1	12
STATISTICS COLLECTOR		1	5
BITMAP MERGE		1	5
BITMAP KEY ITERATION		1	240
TABLE ACCESS FULL	DIM3	1	6
BITMAP INDEX RANGE SCAN	DIM3BX	6	240
TABLE ACCESS BY USER ROWID	FACT	6,000	6,000

Look at the end. When the statistics collector has seen that the threshold has been passed over, it has decided to skip that bitmap branch. This is the third case of adaptive plans: Adaptive Bitmap Pruning. The bitmap branch is good only if it helps to filter a lot of rows. If it's not the case, then it's just an overhead, and it is skipped coming back to the behavior we had at the beginning when star transformation was disabled.

12c In-Memory option

I'm talking about new features, and you probably are tired of long execution plans. So let's try In-Memory:

```
alter table FACT inmemory priority critical;
alter table DIM1 inmemory priority critical;
alter table DIM2 inmemory priority critical;
alter table DIM3 inmemory priority critical;
```

I've plenty of memory and I've set my inmemory_size already. I just have to wait a bit, so that the in-memory column store is filled and run my query again:

Operation	Name	I...	Ex...	Act...	Timeline(...	Me...
SELECT STATEMENT		0	1	1,000		
HASH JOIN		1	1	1,000		288KB
JOIN FILTER CREATE	:BF0000	2	1	1		
MERGE JOIN CARTESIAN		3	1	1		
MERGE JOIN CARTESIAN		4	1	1		
TABLE ACCESS INMEMORY FULL	DIM2	5	1	1		
BUFFER SORT		6	1	1		2KB
TABLE ACCESS INMEMORY FULL	DIM3	7	1	1		
BUFFER SORT		8	1	1		2KB
TABLE ACCESS INMEMORY FULL	DIM1	9	1	1		
JOIN FILTER USE	:BF0000	10	1	100K		
TABLE ACCESS INMEMORY FULL	FACT	11	1	100K		

Of course, you need to have the FACT table in memory, or at least the interesting partition. But then you don't need star transformation. As usual you have that cartesian merge join to get the dimension. But then you remember that, without star transformation, you accessed to the FACT through NESTED LOOP JOIN or HASH JOIN – and that was adaptive.

Here the FACT is stored in memory and there is no index access, so we use a full scan. Do you remember that star transformation was nice because it pushes down the predicates to filter the FACT table earlier? Here we have something else. The critera are pushed with a bloom filter. Because we have read all the dimensions first, then we can build the bloom filter (JOIN FILTER CREATE) and use it (JOIN FILTER USE) to filter a large part of the rows – saving the cost of lots of hash lookups. Hash lookups have to be done only for the few bloom filter false positives. And vector processing, which is the way to scan columnar data, is very efficient with bloom filters.

Conclusion

This completes the trilogy about adaptive plans that appeared in 12c. I'm sure that a refresh about star transformation was not a bad idea. I've worked a lot on datawarehouses and star schemas but still had to study it when preparing the OCM exam. And the adaptive feature in this area has not been widely documented. I've concluded with In-Memory because I think that the star transformation, and especially the bitmap indexes, was a premise of a columnar approach. The problem is that they don't like OLTP updates. You can have star transformation with regular indexes as well, but there is a rowid-to-bitmap transformation that has a big overhead. In-Memory is a good solution for ad-hoc queries on OLTP databases – as long as you have enough memory to keep your data in the in-memory column store. But about that, keep in mind that columnar compression has great ratios on fact tables because of the repeated dimension keys. ■

Contact

dbi services

Franck Pachot
E-Mail:
franck.pachot@dbi-services.com

SMS >>>

Neue Oracle Cloud Services

Oracle hat die Verfolgung der etablierten Cloud Service Provider definitiv aufgenommen und addiert sechs neue Dienste:

- Oracle Big Data cloud – nicht ganz die erste Cloud Lösung eines Hadoop frameworks
- Oracle Mobile Cloud für «Enterprise-grade» Mobile Apps...
- Oracle Integration Cloud verschmelzt Cloud/Cloud und Cloud/OnPrem über ein BUI
- Oracle Process Cloud ermöglicht das einfache Abbilden von Geschäftsprozessen. Oder so.
- Oracle Node.js Cloud für Java Scripter in der Wolke
- Oracle JAVE SE Cloud für JAVA SE Anwendungen

Zusammen mit den bereits verfügbaren Diensten steht somit einem Geschäft eine umfangreiche Sammlung von Möglichkeiten und Diensten zur Verfügung. Mit den breit verfügbaren Trial Möglichkeiten und Preisplänen lohnt sich eine genauere Betrachtung. Wir empfehlen allerdings für diese genaue Betrachtung auch wirklich Zeit zu investieren – an einem Abend ist das nicht erledigt.